



Song Form-aware Full-Song Text-to-Lyrics Generation with Multi-Level Granularity Syllable Count Control

Yunkee Chae¹, Eunsik Shin², Suntae Hwang², Seungryeol Paik², Kyogu Lee^{1,2,3}

¹Interdisciplinary Program in Artificial Intelligence, Seoul National University,

²Department of Intelligence and Information, Seoul National University,

³Artificial Intelligence Institute, Seoul National University, Republic of Korea

{yunkimo95, esshin, iamsuntae1, paik402, kglee}@snu.ac.kr

Abstract

Lyrics generation presents unique challenges, particularly in achieving precise syllable control while adhering to song form structures such as verses and choruses. Conventional line-by-line approaches often lead to unnatural phrasing, underscoring the need for more granular syllable management. We propose a framework for lyrics generation that enables multi-level syllable control at the word, phrase, line, and paragraph levels, aware of song form. Our approach generates complete lyrics conditioned on input text and song form, ensuring alignment with specified syllable constraints. Generated lyrics samples are available at: <https://tinyurl.com/lyrics9999>

Index Terms: lyrics generation, syllable count control

1. Introduction

The field of lyrics information processing [1] presents unique challenges that extend beyond traditional text generation, as lyrics must align with both song form structures (such as verses, choruses, and bridges) and specific syllabic constraints. While natural language generation models have shown promise in generating coherent text, applying these models to lyrics introduces additional complexities due to the need for musical and rhythmic alignment.

Several studies have made notable progress in aligning lyrics with melodies through melody-aligned or midi-aligned generation methods [2, 3, 4]. These efforts have been valuable in addressing melody-lyrics synchronization. However, the scarcity of large-scale audio-lyrics aligned datasets still poses a challenge for these approaches, limiting their generalizability across diverse musical forms [5, 6, 7]. Given this data limitation, we concentrate on a crucial aspect of melody-lyrics alignment: the syllable count, which directly impacts the rhythmic fit of lyrics within a musical composition. However, precise syllable control is challenging, as models like GPT [8], which rely on sub-word tokens, may struggle to manage syllables effectively.

On the other hand, song form—the structured organization of lyrics into verses, choruses, and bridges—plays a crucial role in shaping both the narrative and thematic progression of songs. While some models generate lyrics effectively at a surface level, they often do not account for the intricate patterns imposed by song form, which is essential for producing musically coherent lyrics.

Therefore, we propose a comprehensive full-song lyrics generation system capable of operating under various conditions, including input text, song forms, and syllable counts across different granularities, as illustrated in Figure 1. Trained on a lyrics-only dataset, this system represents the first attempt, to the best of our knowledge, to address these challenges in an all-encompassing manner.

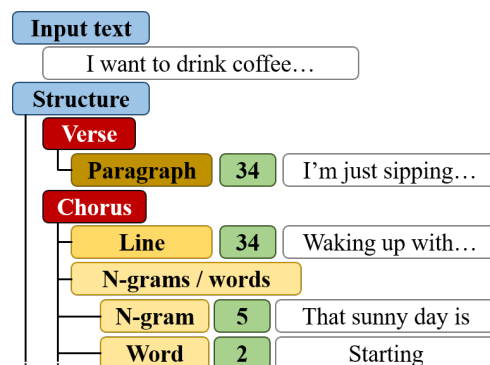


Figure 1: Multi-level granularity lyrics generation framework. The numbers inside the green boxes indicate the syllable count condition.

2. Related Work

Many studies on lyrics generation have attempted to use various features to generate lyrics that match naturally with the melody. Lyrics generation inherits intricate syntactic and semantic challenges from text generation. Notably, within this domain, there has been a significant increase in the utilization of neural networks.

The procedure of converting melodies to expressive lyrics has also been studied, as seen in [3, 4, 9, 10, 11]. These models adeptly generate lyrics in coherence with the provided melody. For instance, [9] used SeqGANs [12] for this task, and [10] proposed AI Lyricists, leveraging MIDI files for lyrics generation. Tailored specifically for melody-to-lyrics generation, models such as [4] utilize architectures like MASS [13]. Though these models address the problem in various ways, they suffer from a lack of melody-lyrics aligned dataset. Approaches used in [11] employ hierarchical frameworks successfully generating lyrics aligned with outlines, trained in an unsupervised manner to address this issue. However, they do not consider song form or multi-granularity conditioned generation.

Researches focused on syllable count control has utilized rule-based and learning-based approaches to produce lyrics that adhere to constraints and convey meaning. Poetry generation is a notable task in this field. For example, studies such as [14, 15, 16] have employed RNN and LSTM for Chinese poetry generation. Additionally, [3] proposed a method that interprets the alignment between lyrics and melodies as representations of syllable structures. This method uses a multi-channel sequence-to-sequence model that considers both phrasal structures and semantics.

(a) *Generation sample*

```

<LYR_START>
<VERSE><SYL:74>
<GEN_P>
There was a time...
<END_P>
<CHORUS><SYL:86>
<GEN_L><SYL:14>
Cause I don't wanna...
<END_L>
<GEN_L_NW><SYL:11>
<GEN_W><SYL:1>Like<END_NW>
<GEN_N><SYL:5>God is a stranger<END_NW>
...
<GEN_N><SYL:2>stranger<END_NW>
...
<END_L>
...
<END_P>

```

(b) *Infilling sample*

```

<COND_START>
<VERSE>
Doesn't matter how you sell me,...<END_L>
<INF_L><SYL:16>
<INF_N><SYL:3> once, <INF_W><SYL:1> me...<END_L>
<CHORUS>
<INF_P><SYL:52>
...
<START>
<VERSE>
<INF_L><SYL:16>Even when...<END_L>
<INF_N><SYL:3> You fool me<END_NW>
<INF_W>fool<END_NW>
<CHORUS>
<INF_P><SYL:52>What's in it...<END_P>
...

```

Figure 2: Training examples of each task. The green tokens represent the tokens directly provided to the model during the inference process, while the red tokens denote the special tokens that the model needs to predict.

3. Methodology

3.1. Multi-Level Granularity Lyrics Generation

Our system generates lyrics at various granularities (words, phrases, lines, paragraphs) with precise syllable control. The model is conditioned on input text, song structure (e.g., verse, chorus), and syllable constraints.

Token Structure. We use a structured token system to align lyrics with song forms, as illustrated in Figure 2a. Each section begins with a song form token (e.g., <VERSE>) followed by a syllable count token (<SYL:s>), where s denotes the designated syllable count for the section, adopting the approach described in [17] and [18]. Inspired by the ideas presented in [19], we implement distinct end tokens for each level of granularity in our model. For paragraphs, the generation starts with <GEN_P> and ends with <END_P>; entire lines use <GEN_L> and <END_L>. For finer segmentation, <GEN_L_NW> tokens are used to indicate the start of such detailed line generation. Each segment, whether phrases or words, begins with <GEN_N> or <GEN_W> tokens, respectively, and concluded with <END_NW> tokens. Upon completing a line detailed with phrases and words, an <END_L> token signifies its end.

Semantic Embedding for Input Text. Our model also incorporates semantic embeddings for the input text, allowing it to generate contextually relevant lyrics. Given the challenge of lacking datasets that pair text with lyrics, we devise an alternative training strategy that does not rely on such paired data. We employ the SentenceTransformer [20] model to capture the semantic content of lyrics, a method inspired by the approach of Watanabe et al. [21], who used Stable Diffusion [22] and pre-trained Vision Transformer [23] for semantic extraction in lyrics. We use semantic embeddings of full lyrics as a substitute condition for the input text during training, enabling the model to generate relevant lyrics by analyzing the semantic content of any input text provided during inference. Specifically, we incorporate the semantic embedding vector into the initial token embedding of the GPT-2 input.

Inference. During inference, the model starts with the semantic embedding from the input text and decodes tokens autoregressive. Special tokens such as song form, syllable counts, and generation directives tokens (<GEN_*>) are provided at decoding step as conditions rather than predicted. The model continues generating until it produces the appropriate end token (<END_*>), then moves on to the next condition, following the complete generation plan.

3.2. Lyrics Infilling

We also propose a lyrics infilling model that can adjust or correct lyrics based on multi-granularity syllable counts, drawing

Models	SCD ↓	SCErr (%) ↓	BERT-S ↑
ChatGPT 3.5	0.363	83.729	0.897
ChatGPT 4	0.194	79.828	0.824
<i>Proposed</i>	0.004	4.396	0.765

Table 1: Comparison with large language models

inspiration from [24]. As in Section 3.1, this method involves the introduction of special tokens designed to mask lyrics at various levels of granularity (e.g., <INF_P>, <INF_L>, etc.), as demonstrated in our infilling examples shown in Figure 2b. We randomly mask parts of the training data by replacing lyrics at different granularities with these tokens, along with their corresponding syllable count tokens. The infilling process begins with the <START> token, followed by the song form tokens and the correct content for the masked segments, each marked with appropriate infilling and syllable tokens.

During inference, the model follows a similar procedure as Section 3.1, with the tokens preceding the <START> token serving as the contextual foundation and generates the missing parts. Special tokens, including song form indicators, infilling directives (e.g. <INF_*>), and syllable counts, are supplied directly to guide the generation, ensuring that each lyrical piece produced matches the specified conditions until the <END_*> is reached.

4. Experiments

4.1. Dataset

For the training and evaluation of our models, we used the Genius Song Lyrics dataset[25], which comprises approximately 5.1 million multilingual song lyrics, with around 3.3 million lyrics in English. From the dataset, we selectively extracted lyrics featuring explicit song form annotations with each paragraph, aligning with a pre-defined set of song forms: verse, chorus, pre-chorus, post-chorus, and bridge.

To address the suitability of the content, we evaluated the toxicity of the lyrics using the Detoxify library [26], excluding any lyrics with a toxicity score greater than 0.5 to foster a positive and inclusive dataset. Syllable counts for each word in the dataset were calculated using the Syllables library [27].

We split the dataset into approximately 340K training, 18K validation, and 10K evaluation samples.

4.2. Preprocessing

Due to the absence of a plan-lyrics paired dataset, we synthesize generation plans from lyric samples. For each training example, we build a hierarchical tree (paragraphs, lines, words) and perform a pre-order traversal, randomly selecting subtrees with probability p , inspired by [24]. For leaf nodes (word), we either include the word directly (50% chance) or select a phrase (ranging from 1 to $\min\{8, \# \text{ of words remaining in the sentence}\}$). At the line level, all syllables are combined as shown in Figure 2. This process informs the model of the required syllable counts in a line in advance. For the infilling task, training samples are created similarly, but selected subtrees are replaced with mask tokens (i.e., <INF_*><SYL:s>) and appended with the target text (see Section 3.2 and Figure 2b). For the evaluation set, since there are no available input text-lyrics pairs, we created input-lyrics pairs by summarizing the lyrics with a summarization model [28].

4.3. Experimental Setup

In all our experiments, we trained the GPT-2 model [29] for 10 epochs, with the batch size of 8, using both scratch and pre-

Model	SCD ↓					SCErr (%) ↓					PPL ↓					BERT-S ↑
	Full	Para.	Line	Phrase	Word	Full	Para.	Line	Phrase	Word	Full	Para.	Line	Phrase	Word	
Front-P	0.026*	0.068*	0.013*	0.021*	0.024*	10.313*	84.856*	11.711*	6.140*	3.618*	17.069	31.259	28.442	16.793	11.271	0.735*
Front-S	0.006*	0.047*	0.004*	0.002	0.002	5.501*	75.781*	3.998*	0.759	0.252	19.036	35.613	31.321	18.377	12.218	0.736*
Both-P	0.016*	0.043*	0.009*	0.012*	0.015*	8.247*	79.516*	8.596*	3.944*	2.178*	17.048	31.652	28.421	16.831	11.259	0.737*
Both-S	0.005*	0.041*	0.003*	0.002	0.001	5.404*	74.847*	3.501*	0.778	0.244	19.453	36.423	31.972	18.847	12.410	0.737*
Back-P	0.014*	0.037*	0.009*	0.011*	0.014*	7.516*	76.813*	7.606*	3.194*	2.041*	17.776	34.667	30.344	17.633	11.847	0.739*
Back-S	0.003	0.027	0.003	0.002	0.002	5.025	68.777	2.851	0.792	0.266	19.883	39.189	33.867	19.422	12.802	0.740

(a) Results of the generation task. Wilcoxon signed-rank tests are conducted between Back-S and others. The average BERT-S between the input text and the original lyrics is 0.799, which serves as an upper bound.

Model	SCD ↓ / SCDerr (%) ↓					PPL ↓ / BERT-S ↑				
	Full	Para.	Line	Phrase	Word	Full	Para.	Line	Phrase	Word
LM-P	0.024*/5.226	0.037/75.763	0.015/7.186	0.009*/2.542*	0.020*/1.493*	7.425 /0.803*	42.030*/0.772*	13.859 /0.746*	5.371 /0.799*	3.037 /0.839*
LM-S	0.002 /3.327	0.034 /72.860	0.004 /4.576	0.001 /0.593	0.001 /0.092	7.887/0.812*	48.166*/0.781*	14.591/0.754*	5.596/0.807*	3.087/0.851*
ILM-P	0.008*/6.642*	0.065*/82.151*	0.016*/15.685*	0.006*/3.282*	0.003*/0.595*	11.675/0.855*	21.021 /0.790*	21.388/0.785*	12.516/0.848*	6.913/ 0.905
ILM-S	0.006/5.479	0.047/75.140	0.013/11.744	0.005/2.331	0.002/0.441	12.897/ 0.856	24.249/ 0.812	23.809/ 0.798	13.829/ 0.850	7.856/0.901

(b) Results of the infilling task. Wilcoxon signed-rank tests are conducted between ILM-S and others.

Note: * $p < 0.005$

Table 2: Results for generation and infilling task. We conducted one-tailed Wilcoxon signed-rank tests comparing each model to the one listed in the bottom row. The alternative hypotheses are that PPL, SCD, and SCDerr are less, while the BERT-S hypothesis is greater.

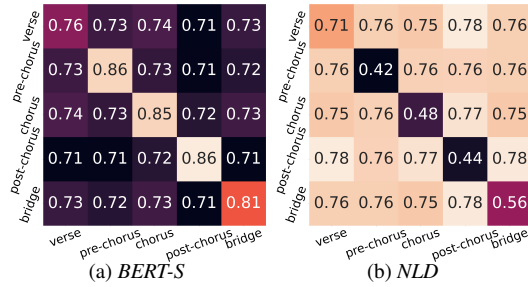


Figure 3: Evaluation of song form consistency. Each metric is calculated between two distinct paragraphs.

trained model. The learning rate was 0.00005 with 500 warm-up steps, and we used the AdamW optimizer [30]. For all experiments, texts are tokenized using a byte-level Byte Pair Encoding (BPE) with a vocabulary size of 50,257 and an input size of 1024 consecutive tokens, as in the original GPT-2 model. For inference, we used top-k (20) and top-p (0.9) sampling with a temperature of 1.0 and a repetition penalty of 1.2. The pre-order tree traversal selection probability p was set to 20% for generation and 10% for infilling.

4.4. Evaluation Metrics

In assessing the performance of our models, we employ a suite of metrics designed to capture various dimensions of model effectiveness, from linguistic predictability to syllabic accuracy and semantic fidelity.

Test set perplexity (PPL) measures the ability of model to predict the text within the test set, providing insights into the model’s linguistic prediction capabilities. PPL is computed on text tokens (excluding special tokens) and reported as a trimmed mean (top 1% excluded).

Syllable count distance (SCD) [31] quantifies the discrepancy between the expected set of syllable counts $S = \{s_1, \dots, s_n\}$ and generated counts $\hat{S} = \{\hat{s}_1, \dots, \hat{s}_n\}$ as:

$$SCD(S, \hat{S}) = \frac{1}{2n} \sum_{i=1}^n (|s_i - \hat{s}_i|/s_i + |s_i - \hat{s}_i|/\hat{s}_i). \quad (1)$$

Syllable count error rate (SCErr) measures how frequently the model generates text with incorrect syllable counts, representing a stricter assessment than SCD by penalizing all inaccuracies equally.

BERT-Score (BERT-S) [32] evaluates semantic coherence by comparing the input text with generated lyrics (or masked seg-

ments with infilled text) using pre-trained BERT embedding. **Normalized Levenshtein Distance (NLD)** is also adopted to assess the consistency of the song form. Levenshtein distance, also known as edit distance, measures the number of character operations (deletions, insertions, or substitutions) required to convert one string into another. Normalized Levenshtein distance [33] is defined as:

$$NLD(p_1, p_2) = LD(p_1, p_2) / \max\{\lambda(p_1), \lambda(p_2)\}, \quad (2)$$

where $LD(p_1, p_2)$ is the Levenshtein distance and $\lambda(p)$ denotes the string length. Metrics are computed overall (Full) and at specific structural levels (e.g., paragraphs) for detailed insights.

5. Results

5.1. Preliminary Experiment

Large language models (LLMs) such as ChatGPT [34] have demonstrated impressive text generation abilities but often struggle with accurate syllable counting [35]. To evaluate this, we asked an LLM to generate a single paragraph based on input text that specified the number of lines and the syllable count per line. A trial was deemed a failure if the model did not produce the correct number of lines within five attempts. ChatGPT 3.5 achieved a success rate of approximately 38%, while ChatGPT 4.0 reached about 57%. Table 1 summarizes the results for 688 successful samples, indicating that while LLMs are capable of generating lyrics that are contextually appropriate to the input text, they perform poorly in matching the required syllable count.

5.2. Lyrics Generation

In Table 2a, we present the performance of our models across various settings. For each training method, we report on models trained from scratch as well as those initialized with large-scale pre-trained GPT-2 weights.

Front models serve as our baseline. In these models, the complete generation plan is provided upfront—before the `<LYR_START>` token—so that the model generates full lyrics without additional guidance during decoding. Essentially, we concatenate all special tokens into a single prompt to generate the output. *Back* models provide the generation plan to the model during decoding, as shown in Figure 2a. *Both* approach merges these methods: it first presents the complete generation plan before the `<LYR_START>` token and then generates the

Model	SCD ↓ / SCErr (%) ↓					PPL ↓ / BERT-S ↑				
	Full	Para.	Line	Phrase	Word	Full	Para.	Line	Phrase	Word
<i>ILM-S</i>	0.006 / 5.444	0.047 / 74.952	0.013 / 11.924	0.005 / 2.405	0.002 / 0.449	12.978 / 0.856	24.158 / 0.813	23.798 / 0.798	13.979 / 0.849	7.869 / 0.900
— — — — —	—	—	—	—	—	—	—	—	—	—
—same mask	0.007 / 5.439	0.052** / 77.601**	0.013 / 12.314*	0.005 / 2.240	0.003** / 0.522*	13.274** / 0.853**	24.277** / 0.813	24.338** / 0.795**	14.357** / 0.845**	7.989** / 0.897**
—no song form	0.006 / 5.318	0.053** / 76.203*	0.012 / 11.771	0.005 / 2.388	0.002 / 0.046	13.213** / 0.855**	24.208** / 0.812	24.014** / 0.798	14.453** / 0.848**	7.919** / 0.899**
—same mask & no song form	0.007** / 5.763**	0.054** / 79.741**	0.014** / 13.834**	0.005 / 2.449	0.003** / 0.551**	13.623** / 0.852**	24.514** / 0.812	24.146** / 0.795**	14.836** / 0.844**	8.588** / 0.895**

Note: * $p < 0.05$, ** $p < 0.005$

Table 3: Results of infilling variants. One-tailed Wilcoxon signed-rank tests are conducted to compare each model with *ILM-S*.

lyrics using the *Back* strategy. The suffixes *-S* and *-P* indicate whether the model was trained from scratch or initialized with pre-trained weights, respectively.

Table 2a shows that pre-trained models achieve lower PPL than models trained from scratch, with the *Front* and *Both* approaches performing best. A high PPL indicates that the model struggles to predict the correct tokens. The reason *Front* and *Both* models exhibit lower PPL compared to *Back* models is likely because the model can preview all conditions before generating lyrics, enabling it to make more informed predictions. In contrast, the *Back* models receive conditioning at each decoding step, limiting their foresight and resulting in higher PPL because they cannot fully anticipate the complete structure. However, given conditions directly at each decoding step could confer advantages in predicting syllable counts. The *Both* models attempt to integrate the strengths of both *Front* and *Back* models but at the expense of increased sample length. Thus, each conditioning approach inherently involves a trade-off.

On the other hand, models trained from scratch consistently outperform pre-trained models in terms of syllable count accuracy, as measured by SCD and SCErr. This may be because scratch models learn to handle special tokens—like syllable tokens—more flexibly. Notably, the *Back-S* model excels in both syllable metrics and BERT-S performance, indicating its superior ability to capture the input text’s semantics by generating immediately after the text embedding.

5.3. Lyrics Infilling

Table 2b presents the results for the infilling task. For baseline comparisons, we use the same models as in the generation experiments—referred to as *LM-S* and *LM-P*—which generate masked segments using only past context (with unmasked tokens provided directly). In contrast, our proposed Infilling Language Models (*ILM-S* and *ILM-P*) leverage both past and future context. While pre-trained models generally achieve lower PPL, models trained from scratch consistently yield better syllable accuracy. *LM* models exhibit lower PPL than *ILMs*, and notably, *LM-S* demonstrates superior syllable count accuracy. These results suggest that leveraging comprehensive context does not necessarily improve syllable count accuracy. However, *ILM-S* attains higher BERT-S scores than its counterparts, indicating that using full context during infilling enhances semantic coherence. Overall, *ILM-S* outperforms *ILM-P* in both syllable metrics and BERT-S, suggesting that training from scratch better adapts to our specialized token structure. These findings are consistent with our observations from the lyrics generation experiments.

5.4. Song Form Consistency Evaluation

To verify that our model generates distinct lyrics for different song form types, we measure semantic similarity (BERT-

S) and structural similarity (normalized Levenshtein Distance, NLD) between paragraphs. We compare only distinct paragraphs within the same song (e.g., verse1 vs. verse2, verse1 vs. chorus1) and compute the average values across all songs.

Figure 3 displays the results for the *Back-S* model in a square matrix format. As these comparisons are made between paragraphs within the same song, we observe that the BERT-S generally exceeds 0.7. Notably, the BERT-S scores for comparisons within the same song form (diagonal values) are higher than those for comparisons between different forms. Similarly, lower NLD values for same-form comparisons indicate that fewer edits are needed between similar segments. Although verses exhibit slightly more variability due to their diverse content, they still show greater similarity within the same form compared to different forms. Overall, these results indicate that the model produces lyrics that are consistent within a given song form and distinct across different forms.

5.5. Ablation Study

Table 3 examines the impact of modifications to our infilling strategy compared to the baseline *ILM-S* model. First, we train a variant using a uniform `<MASK>` token for all conditioning instead of specialized tokens (e.g., `<INF_*><SYL:s>`) for different granularities; we denote this variant as *same mask* in Table 3. In this setup, syllable condition tokens are provided directly at each infilling step. The results show a significant decline in syllable count performance at both the paragraph and word levels, along with reductions in overall BERT-S and increased PPL. Next, we train a model without including song form tokens after the `<START>` token, denoted as *no songform*. While this change has little effect on syllable count accuracy, it also results in a noticeable drop in BERT-S and a rise in PPL, similar to the *same mask* condition. Finally, we combine both modifications—using a uniform mask and omitting song form tokens—in a single model. This configuration degrades performance across nearly all metrics, confirming the importance of our specialized masking strategy and the inclusion of song form annotations for effective lyrics infilling.

6. Conclusion

Our study proposes an effective approach to automated lyrics generation and infilling that leverages multi-level granularity for precise syllable control and effective song form management, all conditioned on an arbitrary input text. This method enhances the flexibility and structural accuracy of the generated lyrics. In future work, we plan to incorporate additional controls—such as genre tags and rhyme schemes—to further improve the adaptability and creative potential of automated lyrics composition.

7. Acknowledgements

This work was supported by the National Research Foundation of Korea (NRF) grant funded by the Korea government (MSIT) [No. RS-2024-00461617, 40%], Information & communications Technology Planning & Evaluation (IITP) grant funded by the Korea government (MSIT) [No. RS-2022-II220320, 2022-0-00320, 50%], [No. RS-2021-II211343, Artificial Intelligence Graduate School Program (Seoul National University), 5%], and [No. RS-2021-II212068, Artificial Intelligence Innovation Hub, 5%]

8. References

- [1] K. Watanabe and M. Goto, "Lyrics information processing: Analysis, generation, and applications," in *Proc. of the 1st Workshop on NLP for Music and Audio (NLP4MusA)*, 2020, pp. 6–12.
- [2] K. Watanabe, Y. Matsubayashi, S. Fukayama, M. Goto, K. Inui, and T. Nakano, "A melody-conditioned lyrics language model," in *Proc. of NAACL-HLT 2018 (Long Papers)*, 2018, pp. 163–172.
- [3] X. Lu, J. Wang, B. Zhuang, S. Wang, and J. Xiao, "A syllable-structured, contextually-based conditionally generation of chinese lyrics," in *Proc. of PRICAI 2019*. Springer, 2019, pp. 257–265.
- [4] Z. Sheng, K. Song, X. Tan, Y. Ren, W. Ye, S. Zhang, and T. Qin, "Songmass: Automatic song writing with pre-training and alignment constraint," in *Proc. of AAAI 2021*, vol. 35, no. 15, 2021, pp. 13 798–13 805.
- [5] G. Meseguer-Brocal, A. Cohen-Hadria, and G. Peeters, "Dali: A large dataset of synchronized audio, lyrics and notes, automatically created using teacher-student machine learning paradigm," *arXiv preprint arXiv:1906.10606*, 2019.
- [6] Y. Yu, A. Srivastava, and S. Canales, "Conditional lstm-gan for melody generation from lyrics," *ACM Transactions on Multimedia Computing, Communications, and Applications (TOMM)*, vol. 17, no. 1, pp. 1–20, 2021.
- [7] S. Durand, D. Stoller, and S. Ewert, "Contrastive learning-based audio to lyrics alignment for multiple languages," in *Proc. of ICASSP 2023*. IEEE, 2023, pp. 1–5.
- [8] T. Brown, B. Mann, N. Ryder, M. Subbiah, J. D. Kaplan, P. Dhariwal, A. Neelakantan, P. Shyam, G. Sastry, A. Askell *et al.*, "Language models are few-shot learners," *Advances in neural information processing systems*, vol. 33, pp. 1877–1901, 2020.
- [9] Y. Chen and A. Lerch, "Melody-conditioned lyrics generation with segans," in *2020 IEEE International Symposium on Multimedia (ISM)*. IEEE, 2020, pp. 189–196.
- [10] X. Ma, Y. Wang, M.-Y. Kan, and W. S. Lee, "Ai-lyricist: Generating music and vocabulary constrained lyrics," in *Proceedings of the 29th ACM International Conference on Multimedia*, 2021, pp. 1002–1011.
- [11] Y. Tian, A. Narayan-Chen, S. Oraby, A. Cervone, G. Sigurdsson, C. Tao, W. Zhao, Y. Chen, T. Chung, J. Huang, and N. Peng, "Unsupervised melody-to-lyrics generation," in *Proc. of ACL 2023 (Long Papers)*, 2023, pp. 9235–9254.
- [12] L. Yu, W. Zhang, J. Wang, and Y. Yu, "Seqgan: sequence generative adversarial nets with policy gradient," in *Proc. of AAAI 2017*, 2017, p. 2852–2858.
- [13] K. Song, X. Tan, T. Qin, J. Lu, and T.-Y. Liu, "MASS: Masked sequence to sequence pre-training for language generation," in *Proc. of ICML 2019*, vol. 97. PMLR, 2019, pp. 5926–5936.
- [14] X. Zhang and M. Lapata, "Chinese poetry generation with recurrent neural networks," in *Proc. of EMNLP*, 2014, pp. 670–680.
- [15] X. Yi, R. Li, and M. Sun, "Generating chinese classical poems with rnn encoder-decoder," in *Chinese Computational Linguistics and Natural Language Processing Based on Naturally Annotated Big Data: 16th China National Conference, CCL 2017, and 5th International Symposium, NLP-NABD 2017, Nanjing, China, October 13-15, 2017, Proceedings 16*. Springer, 2017, pp. 211–223.
- [16] X. Yi, M. Sun, R. Li, and W. Li, "Automatic poetry generation with mutual reinforcement learning," in *Proc. of EMNLP 2018*, 2018, pp. 3143–3153.
- [17] H. Kim, J. Jung, D. Jeong, and J. Nam, "K-pop lyric translation: Dataset, analysis, and neural-modelling," in *Proc. of LREC-COLING 2024*, 2024, pp. 9974–9987.
- [18] F. Guo, C. Zhang, Z. Zhang, Q. He, K. Zhang, J. Xie, and J. Boyd-Graber, "Automatic song translation for tonal languages," in *Findings of ACL 2022*, 2022, pp. 729–743.
- [19] H. Bai, P. Shi, J. Lin, L. Tan, K. Xiong, W. Gao, J. Liu, and M. Li, "Semantics of the unwritten: The effect of end of paragraph and sequence tokens on text generation with GPT2," in *Proc. of ACL-IJCNLP Student Research Workshop 2021*, 2021, pp. 148–162.
- [20] N. Reimers and I. Gurevych, "Sentence-bert: Sentence embeddings using siamese bert-networks," in *Proc. of EMNLP 2019*, 2019, also available at <https://huggingface.co/sentence-transformers/all-mpnet-base-v2>.
- [21] K. Watanabe and M. Goto, "Text-to-lyrics generation with image-based semantics and reduced risk of plagiarism," in *ISMIR 2023 Hybrid Conference*, 2023.
- [22] R. Rombach, A. Blattmann, D. Lorenz, P. Esser, and B. Ommer, "High-resolution image synthesis with latent diffusion models," in *Proc. of CVPR 2022*, 2022, pp. 10 684–10 695.
- [23] A. Dosovitskiy, L. Beyer, A. Kolesnikov, D. Weissenborn, X. Zhai, T. Unterthiner, M. Dehghani, M. Minderer, G. Heigold, S. Gelly *et al.*, "An image is worth 16x16 words: Transformers for image recognition at scale," in *Proc. of ICLR 2021*, 2021.
- [24] C. Donahue, M. Lee, and P. Liang, "Enabling language models to fill in the blanks," in *Proc. of ACL 2020*, 2020, pp. 2492–2501.
- [25] "Genius song lyrics dataset," <https://www.kaggle.com/datasets/carlosgcdj/genius-song-lyrics-with-language-information>.
- [26] L. Hanu and Unitary team, "Detoxify," Github. <https://github.com/unitaryai/detoxify>, 2020.
- [27] "Syllables: A fast syllable estimator for python," <https://pypi.org/project/syllables/>.
- [28] M. Lewis, Y. Liu, N. Goyal, M. Ghazvininejad, A. Mohamed, O. Levy, V. Stoyanov, and L. Zettlemoyer, "BART: Denoising sequence-to-sequence pre-training for natural language generation, translation, and comprehension," in *Proc. of ACL 2020*, 2020, pp. 7871–7880.
- [29] A. Radford, J. Wu, R. Child, D. Luan, D. Amodei, I. Sutskever *et al.*, "Language models are unsupervised multitask learners," *OpenAI blog*, vol. 1, no. 8, p. 9, 2019, pre-trained checkpoints are available at <https://huggingface.co/openai-community/gpt2>.
- [30] I. Loshchilov and F. Hutter, "Decoupled weight decay regularization," in *Proc. of ICLR 2019*, 2019.
- [31] H. Kim, K. Watanabe, M. Goto, and J. Nam, "A computational evaluation framework for singable lyric translation," *ISMIR 2023 Hybrid Conference*, 2023.
- [32] T. Zhang, V. Kishore, F. Wu, K. Q. Weinberger, and Y. Artzi, "BERTscore: Evaluating text generation with bert," *Proc. of ICLR 2020*, 2020.
- [33] K. Tashima, H. Aman, S. Amasaki, T. Yokogawa, and M. Kawahara, "Fault-prone java method analysis focusing on pair of local variables with confusing names," in *2018 44th Euromicro Conference on Software Engineering and Advanced Applications (SEAA)*. IEEE, 2018, pp. 154–158.
- [34] L. Ouyang, J. Wu, X. Jiang, D. Almeida, C. Wainwright, P. Mishkin, C. Zhang, S. Agarwal, K. Slama, A. Ray *et al.*, "Training language models to follow instructions with human feedback," *Advances in neural information processing systems*, vol. 35, pp. 27 730–27 744, 2022.
- [35] J. Sun, Y. Tian, W. Zhou, N. Xu, Q. Hu, R. Gupta, J. Wieting, N. Peng, and X. Ma, "Evaluating large language models on controlled generation tasks," in *Proc. of EMNLP 2023*, 2023, pp. 3155–3168.